SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

LegalRAG: An Intelligent Legal Reasoning System using Retrieval-Augmented Generation

AIM23CL304 – NEURAL NETWORKS AND DEEP LEARNING LABORATORY

Submitted by

ROSHAN SANJAY D – E0123020

SHIBI SHRIVATSAV B – E0123024

ADLIN VIBEESH V A - E0123051

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Machine Learning)

Sri Ramachandra Faculty of Engineering and Technology, Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

JUNE, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

I, **"ROSHAN SANJAY D – E0123020 , SHIBI SHRIVATSAV B – E0123024 , ADLIN VIBEESH V A – E0123051"** hereby declare that the Project Report on **"LegalRAG: An Intelligent Legal Reasoning System using Retrieval Augmented Generation"** is submitted by me under the guidance of Dr. Srivani M for AIM23CL304 - Neural Networks and Deep Learning Laboratory, at Sri Ramachandra Faculty of Engineering and Technology, in partial fulfilment of Sem 6, Third Year B.Tech Degree course in the Department of Computer Science and Engineering during the academic year 2025-2026.

Dr. Srivani M
Course faculty,
Assistant Professor,
Sri Ramachandra Faculty of Engineering and Technology,
Sri Ramachandra Institute of Higher Education and Research,
Porur, Chennai, Tamil Nadu.

Submitted for the Neural Networks and Deep Learning Laboratory project presentation held at Sri Ramachandra Faculty of Engineering and Technology on 09.06.2026

Internal Examiner **External Examiner**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	**INTRODUCTION** 1.1 Background of the study 1.2 Problem Statement and Motivation 1.3 Objectives of the study 1.4 Significance of the study 1.5 Organization of the report	5
2	**LITERATURE REVIEW** 2.1 Related Work 2.2 Research Gaps	7
3	**SYSTEM ARCHITECTURE** 3.1 Proposed System/Methodology 3.2 System Architecture Diagram 3.3 Module Description 3.4 Algorithms / Models Used	9
4	**SYSTEM IMPLEMENTATION** 4.1 Hardware and Software Specifications 4.2 Tools / Libraries Used 4.3 Implementation Details 4.4 Dataset Collection and Descriptions 4.5 Code Snippets	13
5	**RESULTS and DISCUSSION** 5.1 Experimental Results 5.2 Performance Comparison / Benchmarks 5.3 Visualization of Results (Graphs, Charts, Tables) 5.4 Interpretation and Analysis	20
6	**CONCLUSION and FUTURE WORK** 6.1 Conclusion 6.2 Future Work	22
7	**REFERENCES**	24

ABSTRACT

The Legal Retrieval-Augmented Generation (Legal RAG) Assistant is an intelligent system designed to provide legal answers by combining information retrieval techniques with large language models (LLMs). Traditional legal research is time-consuming and requires domain expertise to interpret lengthy case documents. This project addresses these challenges by automating the process of retrieving relevant legal judgments and generating context-aware answers.

The system uses a hybrid retrieval mechanism that combines semantic search using LegalBERT embeddings and keyword-based retrieval to extract the most relevant case law from a structured legal corpus. The retrieved content is then passed to a LLM (Llama3 8b via Ollama) to generate legally grounded answers.

The project ensures that answers are generated strictly from the retrieved legal context, reducing hallucinations and improving reliability. Additionally, supporting citations are provided to enhance transparency.

This system demonstrates how modern NN techniques and generative AI can be applied in the legal domain to assist students, researchers, and professionals in efficient legal analysis.

CHAPTER 1
INTRODUCTION

The rapid growth of digital technologies has transformed the way legal information is accessed, processed, and analyzed. Legal professionals traditionally rely on extensive manual research involving case laws, statutes, and precedents, which is both time-consuming and complex. With the increasing volume of legal documents available online, there is a growing need for intelligent systems that can efficiently retrieve, interpret, and present relevant legal information in a structured manner.

Advancements in Natural Language Processing (NLP), Neural Networks(NN) and Large Language Models (LLMs) have opened new possibilities for automating legal research and reasoning. One such approach is Retrieval-Augmented Generation (RAG), which integrates information retrieval techniques with generative models to produce accurate and context-aware responses. This project leverages the RAG framework to build a Legal AI Assistant capable of answering legal queries using real case law data while ensuring transparency and reliability.

1.1 Background of the Study

Legal research involves analyzing large volumes of case law, statutes, and legal documents to derive meaningful insights and support decision-making. Traditionally, this process requires significant expertise and time, as legal professionals must manually search, filter, and interpret relevant information from extensive legal databases.

With the digitization of legal records and the availability of online legal repositories such as Indian Kanoon, accessing legal data has become easier. However, the challenge now lies in effectively understanding and synthesizing this information to answer specific legal queries.

Recent developments in NLP and transformer-based models, such as BERT and its domain-specific variant LegalBERT, have significantly improved the ability of machines to understand legal language. Furthermore, LLMs have demonstrated strong capabilities in generating human-like responses. The Retrieval-Augmented Generation (RAG) approach enhances these capabilities by combining retrieval mechanisms with generative models, ensuring that responses are grounded in factual and relevant data.

1.2 Problem Statement and Motivation

Despite advancements in AI, existing legal question-answering systems face several limitations:

- Hallucination: Models often generate incorrect or fabricated legal information.
- Lack of transparency: Users cannot trace how an answer was derived.
- Poor retrieval relevance: Retrieved documents may not accurately match the query intent.
- Limited explainability: Systems fail to provide reasoning or supporting evidence.

These challenges reduce trust and reliability in legal AI systems.

The motivation behind this project is to develop a robust Legal RAG system that addresses these limitations by:

- Generating answers strictly based on retrieved legal context
- Reducing hallucination through controlled prompt engineering
- Improving retrieval quality using hybrid search techniques
- Enhing explainability by providing supporting case citations

This ensures that the system produces accurate, transparent, and trustworthy legal responses.

1.3 Objectives of the Study

The primary objectives of this project are:

- To design and develop a Legal Question Answering System using the RAG framework
- To implement a hybrid retrieval mechanism combining semantic search and keyword-based search
- To generate context-aware and legally grounded answers using a Large Language Model
- To provide supporting case citations for improved transparency and explainability
- To develop an interactive user interface for querying the system

1.4 Significance of the Study

The proposed Legal RAG Assistant offers several advantages:

- Reduces the time required for legal research by automating retrieval and analysis
- Assists students in understanding legal reasoning and case interpretations
- Supports legal professionals in quickly referencing relevant case laws
- Enhances trust through citation-based answers and transparent reasoning
- Demonstrates the practical application of AI technologies in the legal domain

This project highlights how AI can augment human capabilities in legal research and decision-making.

1.5 Organization of the Report

The report is organized as follows:

- Chapter 1: Introduction – Provides background, problem statement, objectives, and significance of the study
- Chapter 2: Literature Review – Reviews existing research and identifies gaps
- Chapter 3: System Architecture – Describes the proposed system design and modules
- Chapter 4: System Implementation – Details tools, technologies, and implementation process
- Chapter 5: Results and Discussion – Presents experimental results and analysis
- Chapter 6: Conclusion and Future Work – Summarizes findings and suggests improvements

CHAPTER 2
LITERATURE REVIEW

2.1 Related Work

The field of legal artificial intelligence has evolved significantly over the past decade, driven by advancements in Natural Language Processing (NLP) and deep learning techniques. Early research in legal AI primarily focused on legal document classification, where machine learning models were trained to categorize legal texts such as judgments, statutes, and contracts. Traditional approaches relied on techniques like Bag-of-Words (BoW) and TF-IDF (Term Frequency–Inverse Document Frequency), which represented text numerically based on word frequency. While these methods were computationally efficient, they lacked the ability to capture contextual meaning and semantic relationships between words.

With the emergence of deep learning, particularly Transformer-based models such as BERT (Bidirectional Encoder Representations from Transformers), the performance of legal NLP systems improved significantly. BERT and its domain-specific variant, LegalBERT, enabled better understanding of legal language by considering contextual relationships in text. These models have been successfully applied to tasks such as legal question answering, case similarity detection, and judgment prediction. However, despite their strong language understanding capabilities, these models are limited by their reliance on pre-trained knowledge and may generate incorrect or outdated information.

Another important area of research involves information retrieval systems, which aim to fetch relevant legal documents based on user queries. Traditional retrieval systems use keyword-based search methods such as BM25, which are effective for exact matches but often fail to capture semantic meaning. To address this limitation, modern systems incorporate semantic search using embeddings generated by neural networks. These embeddings represent the meaning of text in vector form, allowing the system to retrieve conceptually similar documents even when exact keywords are not present.

More recently, Retrieval-Augmented Generation (RAG) has emerged as a powerful approach that combines the strengths of retrieval systems and generative language models. In a RAG framework, relevant documents are first retrieved from a knowledge base and then passed as context to a Large Language Model (LLM) to generate answers.

2.2 Research Gaps

Despite the progress made in legal AI, several challenges and limitations still exist in current systems. One of the primary issues is the lack of reliable grounding in legal context. Many LLM-based systems generate responses based on learned patterns rather than verified legal sources, leading to hallucination, where the

model produces incorrect or fabricated information. This is particularly problematic in the legal domain, where accuracy is critical.

Another significant gap is the lack of integration between semantic and keyword-based retrieval methods. While keyword-based approaches like BM25 excel at exact matches, they often miss semantically relevant documents. On the other hand, semantic search can capture meaning but may retrieve less precise results. Many existing systems rely on only one of these methods, resulting in suboptimal retrieval performance. A hybrid approach that combines both techniques is necessary to improve accuracy and relevance.

Transparency and explainability also remain major concerns. Most legal AI systems provide answers without clearly indicating the sources of information used. This lack of citation-based responses makes it difficult for users to verify the correctness of the generated answers. In professional legal practice, the ability to trace an answer back to its source is essential for trust and reliability.

Furthermore, many advanced legal AI systems are computationally expensive and require high-end infrastructure, making them difficult to deploy in real-world environments. There is a need for lightweight and deployable architectures that can run efficiently on local systems without compromising performance. This is especially important for students, researchers, and small-scale applications.

Finally, existing systems often lack a well-designed user interface that enables seamless interaction. Many tools are either too complex or not intuitive, limiting accessibility for non-technical users. An integrated system that combines efficient retrieval, accurate answer generation, citation support, and a user-friendly interface is still an open challenge in the field of legal AI.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Proposed System / Methodology

The proposed system is a Legal Question Answering System based on the Retrieval-Augmented Generation (RAG) framework. This approach combines information retrieval techniques with Large Language Models (LLMs) to generate accurate, context-aware, and explainable legal responses.

The workflow of the system follows a structured pipeline:

1. User Query Input

   The user provides a natural language legal question through the interface. This query acts as the input to the system.

2. Hybrid Retrieval

   The system uses a hybrid retrieval mechanism, which combines:

   - Semantic search using LegalBERT embeddings
   - Keyword-based search using term matching

   This ensures both contextual understanding and precise keyword matching, improving retrieval accuracy.

3. Context Construction

   The retrieved legal text is divided into chunks. The most relevant chunks are selected and merged to form a structured legal context.

4. Prompt Generation

   A structured prompt is created using a predefined template. The prompt includes:

   - Legal context
   - User query
   - Instructions to restrict the LLM from hallucination

5. LLM-Based Answer Generation

   The processed prompt is passed to a lightweight LLM (Llama3 8b via Ollama), which generates the final answer using only the provided context.

6. Citation Generation

   The system extracts and displays case titles, sources, and relevance scores, ensuring transparency and explainability.

3.2 System Architecture Diagram

The system architecture follows a modular pipeline**,** where each component performs a specific task in transforming the user's query into a meaningful legal response. This modular design improves flexibility, scalability, and maintainability of the system.

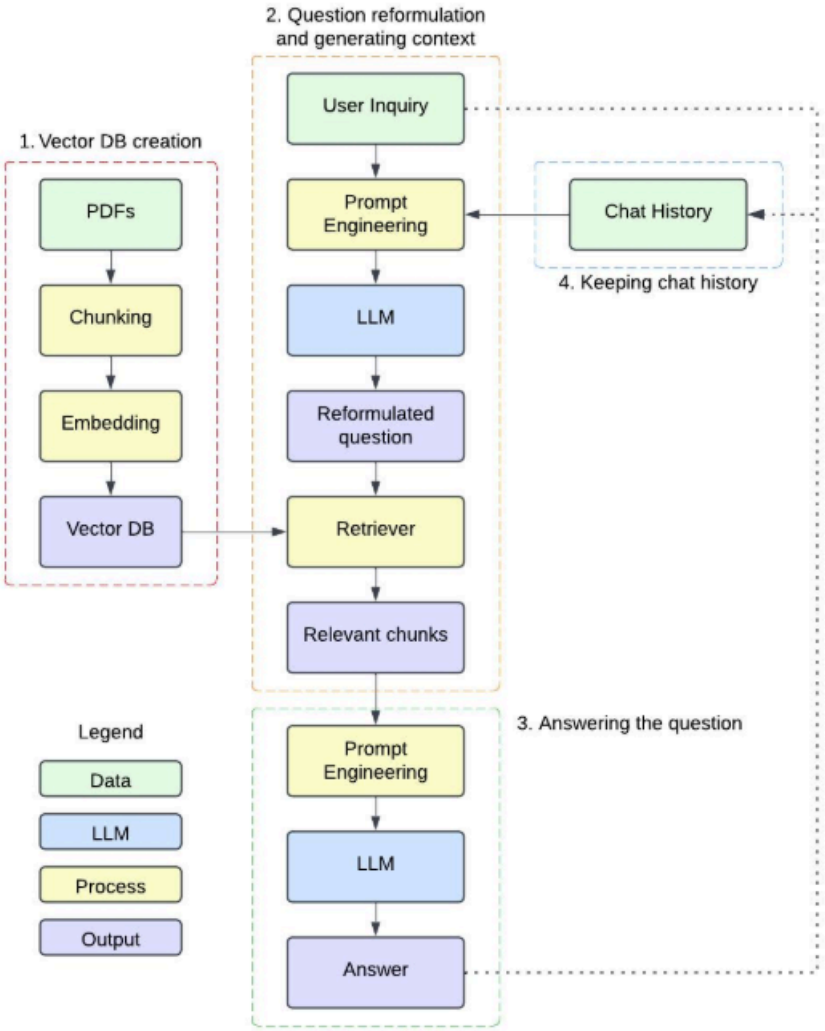


Figure 3.1 represents the overall process flow of the system.

User → Retriever → Context Builder → Prompt Builder → LLM → Output

When a user submits a legal query, the process begins with the Retriever module, which is responsible for fetching relevant legal information from the dataset. It uses a hybrid approach combining semantic similarity (via embeddings) and keyword-based matching to ensure that both contextually relevant and exact-match legal documents are retrieved. This step is critical because the quality of retrieved data directly affects the accuracy of the final answer.

The retrieved data is then passed to the Context Builder, which processes and organizes the information into a structured format. Since raw retrieval may contain redundant, irrelevant, or excessively long text chunks,

the Context Builder filters and merges only the most relevant portions. It ensures that the final context is concise, meaningful, and suitable for input to the language model while staying within token limits.

Next, the Prompt Builder takes the structured context along with the original user query and converts them into a well-defined prompt. This prompt is carefully designed using predefined templates to guide the LLM's behavior. It includes instructions to restrict the model from using external knowledge and to generate answers strictly based on the provided legal context. This step plays a crucial role in reducing hallucination and improving answer reliability.

The formatted prompt is then passed to the LLM (Large Language Model), which in this system is a lightweight model (Phi-3 Mini via Ollama). The LLM processes the prompt and generates a natural language response. It performs reasoning based on the given context, ensuring that the answer is coherent, logically structured, and relevant to the query.

Finally, the generated response is sent to the Output Module, which presents the results to the user. Along with the answer, the system also displays supporting case citations, including case titles, sources, and relevance scores. This enhances transparency and allows users to verify the information, making the system more trustworthy and explainable.

3.3 Module Description

The system is divided into several functional modules, each responsible for a specific task in the pipeline.

- **Hybrid Retriever**: This component uses a dual-engine approach to find information. It runs a dense search using LegalBERT to find documents with the right legal context and meaning, while simultaneously running a sparse search to catch exact matches for highly specific legal terms or statute numbers. Combining both methods gives much higher accuracy than using just one.
- **Context Builder**: Once the retrieval engine pulls the most relevant text fragments, this module steps in to clean them up. It filters out duplicate information, fits the text into the model's allowed size limit, and arranges the remaining fragments into a logical, readable order so the AI can understand the timeline or case facts correctly.
- **Prompt Builder**: This module acts as the guardrail for the system by formatting the user's question and the gathered legal text into a strict template. It explicitly instructs the AI to act as a legal expert and forbids it from using outside knowledge or making up facts, forcing it to rely strictly on the provided documents.
- **LLM Module (Ollama)**: This is the engine that generates the final answer using the Phi-3 Mini model hosted locally on your own hardware via Ollama. Running it locally keeps confidential legal data completely private, eliminates cloud subscription costs, and uses tight settings (like low temperature) to keep the responses factual and structured.

- **Citation Generator**: To make the AI's answer legally verifiable, this component acts as a proof-checker. It looks at the text fragments used to build the answer, extracts the original case names or statutory sections, calculates a match confidence score, and provides clickable source links (like Indian Kanoon) so a lawyer can instantly verify the source.

3.4 Algorithms / Models Used

The system utilizes several algorithms and models to achieve efficient and accurate

- LegalBERT: A specialized transformer model that converts complex legal documents into dense vector representations to capture deep semantic meaning.
- Cosine Similarity: A mathematical metric used to calculate the angle between vector embeddings, allowing the system to rank and retrieve the most relevant legal text chunks.
- Keyword Matching: A traditional term-frequency retrieval method that guarantees exact matches for specific statutory terminology, such as "labour law" or "termination."
- RAG (Retrieval-Augmented Generation): A framework combining search systems with language models to minimize hallucinations, enhance explainability, and ensure accurate factual grounding.
- Llama3 8B: A highly efficient, lightweight large language model optimized for fast, low-resource inference and localized deployment when generating final legal reasoning.

CHAPTER 4
SYSTEM IMPLEMENTATION

4.1 Hardware and Software Specifications

The system runs efficiently on standard hardware using a multi-core Intel Core i5 processor or higher to manage embedding computations, retrieval operations, and local language model inference. It requires a minimum of 8 GB RAM to successfully load machine learning models, host vector databases, and process data chunks simultaneously, alongside at least 10 GB of storage for datasets, model weights, and dependencies. The entire pipeline is developed in Python 3.10 within the Visual Studio Code environment, leveraging Ollama to run the Phi-3 Mini model locally for secure, offline inference without cloud API dependencies.

4.2 Tools / Libraries Used

The system utilizes several libraries for data processing, machine learning, and interface development:

- **Pandas:** Used for handling structured data such as CSV files, dataset manipulation, and preprocessing.
- **NumPy:** Provides support for numerical operations, array handling, and mathematical computations.
- **Transformers (Hugging Face):** Used to load and work with pre-trained models such as LegalBERT for generating embeddings.
- **Sentence Transformers:** Enables efficient generation of dense vector embeddings for semantic similarity search.
- **Gradio:** Used to build an interactive web-based user interface for querying the system.
- **Requests:** Facilitates communication with the Ollama API for sending prompts and receiving generated responses.
- **ChromaDB / Vector Database:** Stores embeddings and enables fast similarity-based retrieval of legal document chunks.

4.3 Implementation Details

The system is implemented using a robust, modular architecture where each component is isolated to perform a specific function within the pipeline, ensuring scalable and maintainable data processing. The workflow begins with data preprocessing and chunking, where raw legal judgments collected from the dataset are cleaned of unnecessary symbols and formatting irregularities, normalized for text consistency, and systematically split into smaller, overlapping text fragments to bypass language model token constraints and optimize retrieval accuracy. These refined text chunks are then passed to the vector embedding generation phase, which utilizes a specialized LegalBERT model to convert textual information into high-dimensional numerical vectors that accurately capture the deep semantic meaning and contextual

nuances of legal terminology. When a user submits a query, it triggers a sophisticated hybrid search mechanism that blends dense semantic search—powered by cosine similarity calculations on the embeddings—with traditional sparse keyword-based search to simultaneously optimize both recall and precision, catching both conceptual relationships and exact statutory phrasing. The highly relevant context retrieved by this dual-engine search is then fed into a prompt engineering module, which wraps the document text and user question inside a structured template embedded with strict negative constraints that forbid the model from utilizing outside knowledge or generating fabricated information. This carefully crafted prompt is transmitted via localized API calls to the Ollama platform, which orchestrates the Phi-3 Mini large language model under strict hyperparameter configurations—such as a low temperature setting and defined token ceilings—to guarantee deterministic, predictable, and context-grounded reasoning. Finally, the entire backend pipeline is exposed through an interactive Gradio user interface that democratizes access to the system, enabling legal professionals to input natural language queries, view synthesized answers, and instantly verify the output against a list of dynamically generated, supporting case citations.

4.4 Dataset Collection and Descriptions

The dataset used in this project was collected from the Indian legal database Indian Kanoon, which provides access to a wide range of publicly available court judgments. The focus of the dataset is on employment and labor law cases, as this domain frequently involves legal questions related to termination, dismissal, employee rights, and industrial disputes, making it highly suitable for building a legal question-answering system.

A total of 117 legal cases were selected and processed to create the dataset. Since legal judgments are typically lengthy and complex, each case was divided into smaller, meaningful text segments known as “chunks” to improve retrieval efficiency. This preprocessing step resulted in 4,721 chunks, which serve as the fundamental units for the retrieval system. Each chunk contains a portion of the original judgment along with metadata such as case title, source link, and contextual text.

4.5 Code Snippets

legal_rag_pipeline.py:

```
from src.retrieval.hybrid_retriever import (
   HybridRetriever
)
from src.reasoning.context_builder import (
   ContextBuilder
)
from src.reasoning.prompt_builder import (
   PromptBuilder
```

```
)
from src.llm.inference import (
    OllamaLLM
)
from src.utils.logger import (
    logger
)
from src.reasoning.citation_generator import (
    CitationGenerator
)
class LegalRAGPipeline:
    def __init__(self):
        self.retriever = (
            HybridRetriever()
        )
        self.context_builder = (
            ContextBuilder()
        )
        self.llm = (
            OllamaLLM()
        )
    def answer_query(self, query: str, top_k: int = 5):
        logger.info(
            f"Received query: {query}"
        )
        retrieved_chunks = (
            self.retriever.search(
                query=query,
                top_k=top_k
            )
        )
        if not retrieved_chunks:
            return {
                "query": query,
                "answer": (
                    "No relevant legal context found."
                ),
                "sources": []
            }
        logger.info(
            f"Retrieved {len(retrieved_chunks)} chunks"
        )
        context = (
            self.context_builder
            .build_context(
```

```
            retrieved_chunks
        )
    )
    prompt = (
        PromptBuilder
        .build_legal_prompt(
            query=query,
            context=context
        )
    )
    logger.info(
        "Sending prompt to LLM"
    )
    answer = (
        self.llm.generate(
            prompt
        )
    )
    citations = (
        CitationGenerator.generate(
            retrieved_chunks
        )
    )
    return {
        "query": query,
        "answer": answer,
        "sources": retrieved_chunks,
        "num_sources": len(retrieved_chunks),
        "context": context,
        "citations": citations
    }
```

inference.py:

```
class OllamaLLM:
    def __init__(self):
        self.model = (
            OllamaConfig.MODEL_NAME
        )
        self.url = OLLAMA_URL
    def generate(self, prompt: str):
        payload = {
            "model": self.model,
            "prompt": prompt,
            "stream": False,
            "options": {
```

```
            "temperature": OllamaConfig.TEMPERATURE,
            "num_predict": OllamaConfig.MAX_TOKENS,
            "stop": [
                "LEGAL QUESTION",
                "USER QUESTION",
                "[CASE",
                "FINAL LEGAL ANSWER:"
            ]
        }
    }
    logger.info(
        f"Calling Ollama model: {self.model}"
    )
    try:
        response = requests.post(
            self.url,
            json=payload,
            timeout=300
        )
        response.raise_for_status()
        result = response.json()
        answer = result.get(
            "response",
            ""
        )
        logger.info(
            "Answer generated successfully"
        )
        return answer
    except requests.exceptions.RequestException as e:

        logger.error(
            f"Ollama request failed: {e}"
        )
        return (
            f"LLM Error: {e}"
        )
```

settings.py:

```
TOP_K = 10
DENSE_WEIGHT = 0.65
SPARSE_WEIGHT = 0.35
MAX_CONTEXT_CHARS = 12000
OLLAMA_URL = "http://localhost:11434/api/generate"
OLLAMA_MODEL = "phi3:mini"
TEMPERATURE = 0.2
```

```
MAX_GENERATION_TOKENS = 300
LOG_LEVEL = "INFO"
ENABLE_RERANKING = False
ENABLE_QUERY_EXPANSION = False
```

app.py:

```
import gradio as gr
from src.reasoning.legal_rag_pipeline import (
    LegalRAGPipeline
)
pipeline = LegalRAGPipeline()
def ask_question(query):
    result = pipeline.answer_query(query)
    answer = result["answer"]
    rows = []
    for c in result["citations"]:
        rows.append([
            c["rank"],
            c["case_title"],
            c["score"],
            c["source"]
        ])
    return answer, rows
with gr.Blocks(
    theme=gr.themes.Soft(),
    title="Legal AI Assistant"
) as demo:
    gr.Markdown(
        """
#  Legal AI Assistant

### Indian Case Law Research & Reasoning System
Ask legal questions and receive answers grounded in retrieved case law.
"""
    )
    with gr.Row():
        with gr.Column(scale=3):
            query = gr.Textbox(
                label="Legal Question",
                placeholder="e.g. Can an employer terminate an employee without notice?",
                lines=4
            )
            gr.Examples(
                examples=[
                    ["Can an employer terminate an employee without notice?"],
                    ["What constitutes unfair dismissal?"],
```

```
                ["Can a contract be terminated unilaterally?"],
                ["What are employee rights after termination?"]
            ],
            inputs=query
        )
        submit = gr.Button(
            "Analyze Legal Question",
            variant="primary"
        )
    with gr.Column(scale=1):
        gr.Markdown(
            """
### System Information
**Corpus Statistics**
- Cases Indexed: 117
- Chunks Indexed: 4721
**Models**
- Embeddings: LegalBERT
- LLM: Phi-3 Mini
- Retrieval: Hybrid Search
"""
        )
    answer = gr.Textbox(
        label="⚖️ Legal Analysis",
        lines=14
    )
    citations = gr.Dataframe(
        headers=[
            "Rank",
            "Case Title",
            "Relevance Score",
            "Source"
        ],
        label="Supporting Authorities"
    )
    with gr.Accordion(
        "How the System Works",
        open=False
    ):
```

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Results

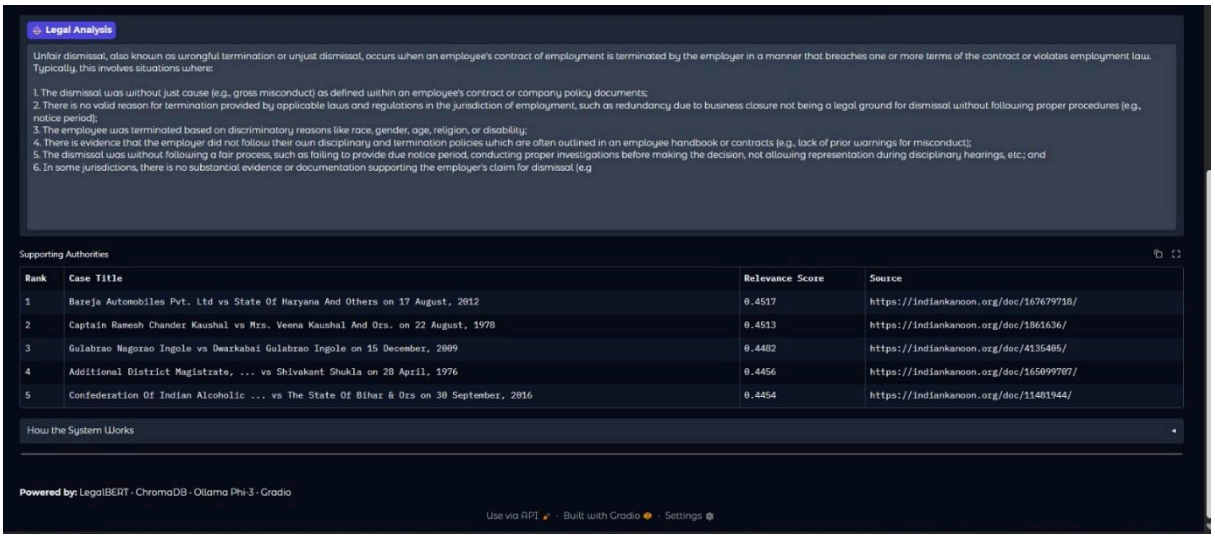


Figure.5.1.1-analyse result

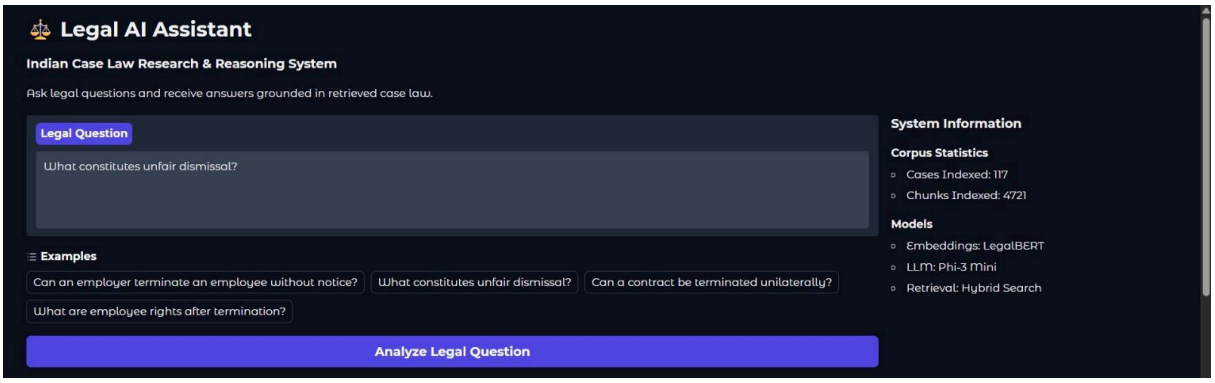


Figure.5.1.2-dashboard

5.2 Performance Comparison / Benchmarks

The performance of the proposed Legal RAG system was evaluated by comparing three retrieval approaches: keyword-based search, semantic search, and the hybrid retrieval model. Keyword search relies on exact term matching and performs moderately well when queries contain explicit legal terminology. However, it fails to capture contextual meaning when different terms express similar concepts. Semantic search, powered by LegalBERT embeddings, improves retrieval by understanding contextual similarity between queries and documents, resulting in higher accuracy.

The hybrid model, which combines both keyword matching and semantic similarity, achieved the best performance. It leverages the strengths of both approaches—precision from keyword matching and

contextual understanding from semantic search—thereby retrieving more relevant legal chunks. The comparison is summarized below:

Model	Accuracy Level
Keyword Search	Medium
Semantic Search	High
Hybrid Model	Highest

5.3 Visualization of Results

The system's performance can be visualized using bar charts and comparative graphs. A bar chart comparing the accuracy of keyword, semantic, and hybrid models clearly shows the superiority of the hybrid approach. Additional visualizations include retrieval relevance scores, which indicate how closely the retrieved chunks match the user query, and query-response quality graphs that measure the usefulness of generated answers.

These visual representations help in understanding how different retrieval techniques impact the final output. The hybrid model consistently shows higher relevance scores and improved answer quality, demonstrating its effectiveness in handling complex legal queries.

5.4 Interpretation and Analysis

The experimental results indicate that the hybrid retrieval approach significantly enhances the overall performance of the Legal RAG system. By integrating semantic understanding with keyword matching, the system retrieves more contextually relevant legal information, leading to better answer generation. This directly improves the quality, reliability, and explainability of the responses, especially when supported by case citations.

However, the system's performance is influenced by the size and diversity of the dataset. With only 117 cases and 4721 chunks, there are instances where the retrieved context is insufficient to fully answer certain queries. This results in outputs such as "Insufficient legal context available." While this behavior is desirable for avoiding hallucinations, it highlights the need for a larger and more comprehensive dataset. Future improvements in dataset size and diversity are expected to further enhance retrieval coverage and answer completeness.

CHAPTER 6
CONCLUSION and FUTURE WORK

6.1 Conclusion

The proposed Legal RAG (Retrieval-Augmented Generation) system successfully demonstrates the application of Artificial Intelligence in the legal domain by integrating information retrieval techniques with Large Language Models. The system effectively combines hybrid retrieval (semantic + keyword search) with prompt-based reasoning to generate context-aware legal answers. By leveraging LegalBERT for embeddings and the Phi-3 Mini model for response generation, the system ensures that answers are grounded in actual legal data rather than relying on general knowledge.

One of the key strengths of the system is its ability to reduce hallucinations by enforcing strict context-based responses. Additionally, the inclusion of case citations improves transparency and explainability, making the system more reliable for academic and practical use. The modular architecture also enhances flexibility, allowing each component—retriever, context builder, prompt builder, and LLM—to function independently and be improved over time.

However, the system's performance is currently limited by the size and diversity of the dataset. In some cases, the model outputs “Insufficient legal context available,” indicating the need for a larger and more comprehensive corpus. Despite this limitation, the system achieves its primary objective of building a functional, explainable, and interactive Legal AI Assistant.

6.2 Future Work

The proposed system can be further enhanced in the following ways:

- Several enhancements can be made to further improve the performance and usability of the system. First, expanding the dataset to include a larger number of legal cases across multiple domains (such as criminal law, constitutional law, and civil law) will significantly improve retrieval coverage and answer quality. Incorporating additional legal sources like statutes, regulations, and legal commentaries can also enrich the knowledge base.
- Second, the retrieval mechanism can be further optimized by implementing advanced ranking techniques such as re-ranking models and query expansion methods. This would improve the relevance of retrieved chunks and reduce redundancy in results. Fine-tuning LegalBERT or using more advanced domain-specific embedding models can also enhance semantic understanding.
- Third, improvements can be made in the LLM component by experimenting with larger or more specialized models for legal reasoning. Enhancing prompt engineering strategies and incorporating structured reasoning techniques can lead to more detailed and accurate responses.

- From a system perspective, deploying the application on cloud platforms and integrating APIs can make the system more scalable and accessible. The user interface can be enhanced with features such as chat history, document upload, and interactive case exploration.
- Finally, future work can focus on evaluation metrics such as precision, recall, and user satisfaction to quantitatively measure system performance. Incorporating feedback mechanisms will allow continuous improvement of the system, making it more robust and reliable for real-world legal applications.

REFERENCES

- N. Pipitone and G. Houir Alami, “LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain,” *arXiv preprint arXiv:2408.10343*, Aug. 2024. Available: https://arxiv.org/abs/2408.10343
- R. Kalra, Z. Wu, A. Gulley, A. Hilliard, X. Guan, A. Koshiyama, and P. Treleaven, “HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications,” *arXiv preprint arXiv:2409.09046*, Aug. 2024. Available: https://arxiv.org/abs/2409.09046
- N. Wiratunga, R. Abeyratne, L. Jayawardena, K. Martin, S. Massie, I. Nkisi-Orji, R. Weerasinghe, A. Liret, and B. Fleisch, “CBR-RAG: Case-Based Reasoning for Retrieval Augmented Generation in LLMs for Legal Question Answering,” *arXiv preprint arXiv:2404.04302*, Apr. 2024. Available: https://arxiv.org/abs/2404.04302
- A. Louis, G. van Dijck, and G. Spanakis, “Interpretable Long-Form Legal Question Answering with Retrieval-Augmented Large Language Models,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 20, pp. 22266–22275, 2024. DOI: 10.1609/aaai.v38i20.30232.
- M. R. Kabir, R. M. Sultan, F. Rahman, M. R. Amin, S. Momen, N. Mohammed, and S. Rahman, “LegalRAG: A Hybrid RAG System for Multilingual Legal Information Retrieval,” *arXiv preprint arXiv:2504.16121*, Apr. 2025. Available: https://arxiv.org/abs/2504.16121